

Concurrent Database Access — Student 3 (Admin / Instructor / Jandric)

Section 4.4 write-up for admin and instructor conflict scenarios in FishyFinds.

Relevant code: `ComplaintController`, `ComplaintService`, `AccountDeletionRequestService`, `LoyaltyProgramController`, `LoyaltyProgramService`, instructor booking with `Term` `@Version`.

1. Two admins answering the same complaint

Situation

Two administrators accept or deny the same pending complaint concurrently, risking double responses or overwritten status.

Endpoints / classes

- `ComplaintController`: `acceptComplaint`, `denyComplaint`, `allPendingComplaints`
- `ComplaintService.acceptComplaint` / `denyComplaint`

Lock approach

Optimistic status check: update only if status is still `PENDING`; optionally add `@Version` on `Complaint`.

Why chosen

Complaint resolution is a single-row status transition; only one admin should win.

2. Two admins answering the same deletion request

Situation

Two admins accept/deny the same account deletion or creation request, causing conflicting emails or inconsistent status.

Endpoints / classes

- `AccountDeletionRequestService.updateRequest` , `denyDeleteRequest`
- `approveCreationRequest` / `denyCreationRequest`

Lock approach

Conditional update from `PENDING` / `PENDING_CREATION` only; second admin aborts if status already terminal.

Why chosen

Status machine naturally serializes decisions without long locks during mail sending.

3. Instructor action versus client reserve

Situation

An instructor defines a quick action or term on a course while a client books overlapping capacity.

Endpoints / classes

- Instructor: `POST /api/createQuickAction` → `ReservationController.createQuickAction` → `ReservationService.createQuickAction`
- `TermService` / related reservation APIs
- `ReservationService.makeReservation` / `makeReservationAction`
- `Term` and `Reservation` with `@Version`

Note on `createQuickAction`

Instructors share `createQuickAction` with owners: concurrent `createQuickAction` vs client `makeReservation` can both observe free capacity and commit overlapping ACTIVE reservations. Rely on `@Version` on `Reservation` / `Term` (plus transactional overlap checks) so only one writer succeeds.

Lock approach

Optimistic locking via `@Version` on `Term` and `Reservation`.

Why chosen

Same shared state as owner/client races; existing version columns apply.

Extra: two admins deleting the same loyalty category

Situation

Two admins call `deleteLoyaltyCategory` for the same id; create path may also race on category name.

Endpoints / classes

- `LoyaltyProgramController` → `LoyaltyProgramService.deleteLoyalty` / `addNewLoyalty`
- `checkIfExists` uniqueness on create

Lock approach

Idempotent delete by id; unique category name (and DB unique constraint recommended) on create.

Why chosen

Second delete is a no-op if the row is gone; name checks reduce duplicate categories.

Summary

Scenario	Primary classes	Control
Same complaint	<code>ComplaintController</code> , <code>ComplaintService</code>	Conditional status / version
Same deletion request	<code>AccountDeletionRequestService</code>	Conditional status
Instructor vs client	<code>ReservationService</code> , <code>Term</code>	<code>@Version</code>
Loyalty delete	<code>LoyaltyProgramController</code>	Idempotent delete